

ระบบค้นคืนสารสนเทศ โดยใช้ฟัซซี่ ลอจิก
Information Retrieval System using Fuzzy Logic

คู่มือการเขียนโปรแกรม

โดย

นายพิศักดิ์	ครูเสถียร
นายพุดตรี	ธาราธิคุณเดช

อาจารย์ที่ปรึกษา

รศ.ดร. เอื้อน ปิ่นเงิน

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต

ภาควิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

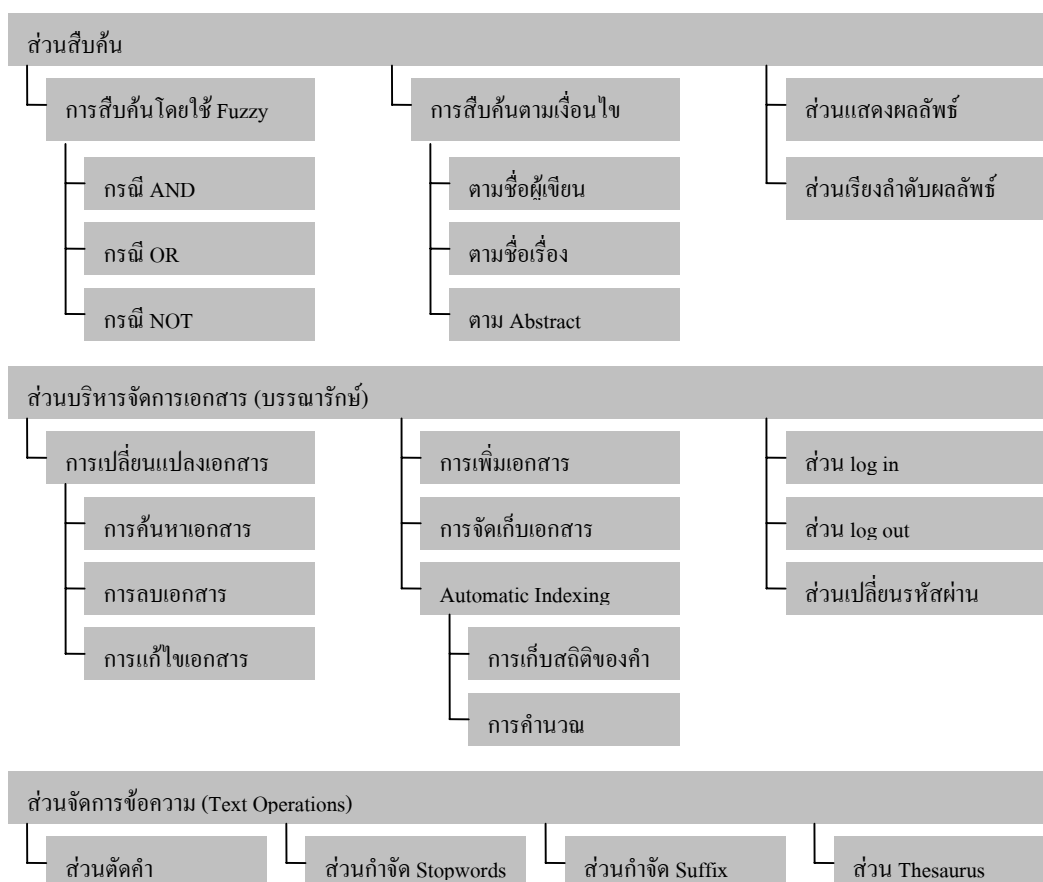
ปีการศึกษา 2544

โครงสร้างของระบบ

ระบบค้นคืนสารสนเทศที่สร้างขึ้น แบ่งออกได้เป็น 2 ส่วน ได้แก่

1. ส่วนของผู้สืบค้นที่จะทำการสืบค้นข้อมูล
2. ส่วนของบรรณารักษ์ที่จะทำการแก้ไข เพิ่มเติมข้อมูล

ซึ่งทั้ง 2 ส่วนนี้ แต่ละส่วนจะมีกลุ่มผู้ใช้ที่แตกต่างกัน ซึ่งมุมมองของผู้ใช้ที่มีต่อระบบก็แตกต่างกัน และแต่ละส่วนก็มีลักษณะหน้าที่การทำงานเฉพาะของตนเอง โดยจะมีเพียงส่วนของฐานข้อมูลเท่านั้นที่ใช้ร่วมกัน



รูปแสดง Hierarchy Chart ของโครงสร้างระบบ

สำหรับส่วนจัดการข้อความนั้น เป็นส่วนที่ถูกเรียกใช้ทั้งในส่วนสืบค้น และส่วนจัดการเอกสาร จึงถูกแยกออกมาเป็นส่วนเฉพาะ ซึ่งรวบรวมฟังก์ชันต่างๆ ในการจัดการกับข้อความไว้ และถูกสร้างในรูปแบบของ Java Bean ในขณะที่ส่วนอื่นจะถูกสร้างในรูปแบบ JavaServer Pages (JSP) ซึ่งจะเรียกใช้ส่วนของ Bean อีกทีหนึ่ง

ส่วนสืบค้น

เป็นส่วนที่ผู้ทั่วไปทำการสืบค้นข้อมูล โดยส่วนการสืบค้นจาก Index terms จะเป็นส่วนที่ใช้ทฤษฎี Fuzzy เข้ามาช่วยในการหาผลลัพธ์ และส่วนการสืบค้นโดยกำหนดเงื่อนไข จะเป็นการสืบค้นตามรูปแบบปกติ

ส่วนสืบค้นปกติ

เป็นการสืบค้นจาก Index terms โดยใช้หลักการ Fuzzy Logic มาประกอบ โดยการสร้างความสัมพันธ์ระหว่างคำที่เป็น Index terms แต่ละคู่ว่ามีความสัมพันธ์(เกี่ยวข้อง)เป็นเท่าไร ซึ่งค่าความสัมพันธ์นี้จะมีค่าอยู่ระหว่าง 0 – 1 ซึ่งค่าความสัมพันธ์นี้จะได้มาจากการคำนวณจากจำนวนเอกสารที่คำคู่ นั้นเป็น Index terms อยู่ โดยค่าความสัมพันธ์เหล่านี้จะถูกคำนวณทุกครั้งที่มีการเปลี่ยนแปลง Index terms ของเอกสาร และจะถูกเก็บอยู่ในตาราง Relation

การหาความสัมพันธ์ระหว่างคำเหล่านี้ คล้ายกับหลักในการสร้าง Thesaurus หรือคำเหมือน แต่แทนที่จะเป็นคำที่เหมือนกัน 100 % เราใช้หลักการของ Fuzzy เข้ามาช่วยบอกเปอร์เซ็นต์ความเกี่ยวข้องกันระหว่างคำแทน โดยคำนวณจากปริมาณเอกสารที่คำคู่ นั้นเป็น Index terms อยู่ ซึ่งเราอาจกล่าวได้ว่าเราพยายามสร้าง “Dynamic Thesaurus ของคำที่เกี่ยวข้องกัน” นั่นเอง

เมื่อเรารู้ความสัมพันธ์ระหว่างคำคู่หนึ่งๆแล้ว เราก็สามารถหาความสัมพันธ์ระหว่างเอกสารกับคำได้ โดยนำค่าความสัมพันธ์ระหว่างคำของ Index terms ทั้งหมดในเอกสารนั้น มารวมกัน

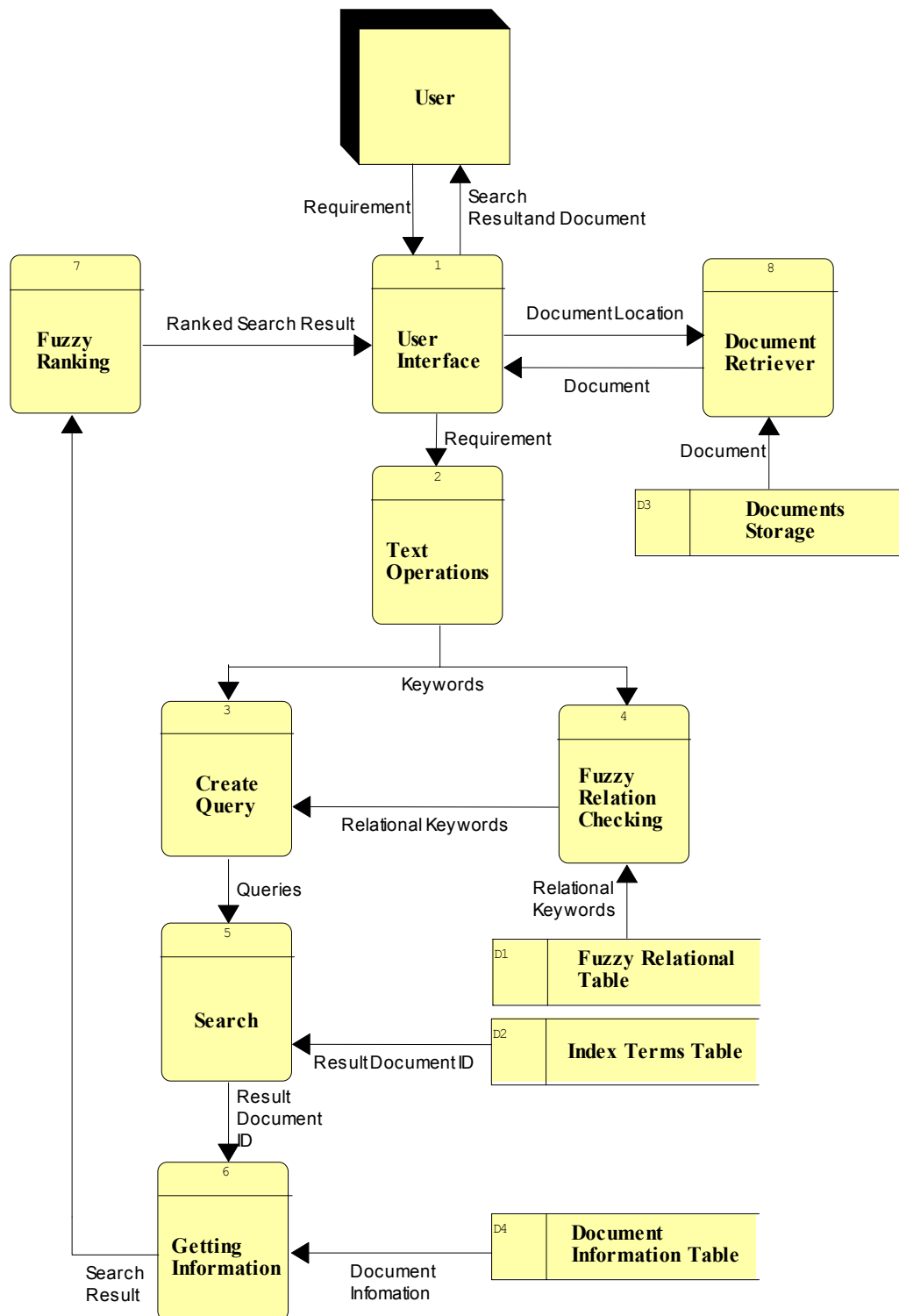
จากหลักการดังกล่าว เมื่อเราต้องการทำการสืบค้น เราจึงต้องทำตามขั้นตอน ดังนี้

1. ทำขั้นตอนจัดการข้อความ เพื่อให้ได้คำที่อยู่ในรูปแบบที่ใช้สืบค้นได้
2. หาคำที่มีความสัมพันธ์กับคำที่ใช้ในการสืบค้น ($\text{Relation} > 0$)
3. นำคำที่ได้จากข้อ 1 มาสืบค้นเอกสาร
4. คำนวณค่าความสัมพันธ์ระหว่างเอกสารที่ได้กับคำที่ใช้ในการสืบค้น โดยอาศัยค่าความสัมพันธ์ที่ได้จากข้อ 1 ของทุกๆคำที่เป็น Index terms ของเอกสารนั้นๆ
5. เก็บคำที่ได้ไว้เพื่อนำไปใช้ในการเรียงลำดับต่อไป

นอกจากนี้ จากหน้าจอ Advance Search จะมีส่วนที่สามารถเลือกลักษณะความเกี่ยวข้องกันระหว่างกลุ่มของคำที่ใช้ในการสืบค้นได้ 3 แบบ คือ AND , OR และ NOT ซึ่งตามปกติแล้วการสืบค้นธรรมดาจะมีค่าเป็น AND กันทั้งหมด โดยทั้ง 3 แบบจะมีวิธีการทำงานที่แตกต่างกัน คือ

- AND เป็นการคำนวณจากกลุ่มคำทั้งหมด ดังนั้นหากคำใดคำหนึ่งในกลุ่มคำที่ใช้สืบค้นไม่เป็น Index terms ของเอกสารนั้นๆ จะส่งผลทำให้ค่าความถูกต้อง (Fuzzy) ที่คำนวณได้มีค่าไม่ถึง 100%
- OR เป็นการคำนวณจากกลุ่มคำกลุ่มใดกลุ่มหนึ่ง ดังนั้นหากคำใดกลุ่มคำใดกลุ่มคำหนึ่งไม่เป็น Index terms ของเอกสารนั้น แต่คำทุกคำของอีกกลุ่มคำหนึ่งเป็น Index terms ของเอกสาร ค่าความถูกต้องที่ได้จะยังคงเป็น 100%

- NOT เป็นการคำนวณแบบเดียวกับ AND แต่ในค่าความสัมพันธ์ที่เป็น Invert แทน ในกลุ่มคำที่อยู่ในเงื่อนไข NOT



รูปแสดง Data Flow Diagram ของการสืบค้นแบบฟัซซี่

สำหรับสูตรและสมการต่างๆที่ใช้ในการคำนวณสามารถหาได้จากปริญญานิพนธ์

ส่วนสืบค้นตามเงื่อนไขอื่นๆ

ในกรณีที่ผู้ใช้ไม่ต้องการค้นหาจาก Index terms แต่ต้องการค้นหาจากข้อมูลประกอบอื่นๆของเอกสาร เช่น ชื่อผู้แต่ง ชื่อเรื่อง ก็สามารถทำได้ ซึ่งกรณีนี้จากมีการทำงานแยกออกมาเป็นส่วนเฉพาะ ซึ่งสามารถเรียกใช้ได้จากหน้าจอ Advance Search ส่วนล่าง โดยแบ่งการค้นหาออกเป็น

- ตามชื่อผู้แต่ง (Author)
- ตามชื่อเรื่อง (Title)
- ตามบทคัดย่อ (Abstract)

ซึ่งทั้งหมดจะเป็นการค้นหาแบบ Pattern matching คือ สามารถค้นหาส่วนของข้อความได้โดยไม่ต้องใส่ข้อความทั้งหมดให้ถูกต้อง เช่น “Comp” จะถือว่าตรงกับ “Computer” , “Compare” และ “Decompression” เป็นต้น ซึ่งทำได้โดยการใช้ LIKE ‘%ข้อความ%’ แทน = ‘ข้อความ’ ในประโยค Select ของภาษา SQL

ในกรณีที่ระบุเงื่อนไขในการค้นหามากกว่า 1 เงื่อนไข จะนำเงื่อนไขทั้งหมดมารวมกัน (AND) ซึ่งในส่วนนี้จะเป็นการค้นหาแบบธรรมดา โดยจะไม่มีการใช้ Fuzzy Logic ดังนั้นหากเงื่อนไขใด เงื่อนไขหนึ่งไม่เป็นจริงก็จะทำให้เอกสารนั้นไม่ถูกนำมาแสดงผล

ส่วนเรียงลำดับผลลัพธ์ (Ranking)

เป็นส่วนที่ทำการเรียงลำดับผลลัพธ์ตามค่าความถูกต้อง (Fuzzy) ที่คำนวณได้ โดยค่าที่คำนวณได้จะถูกเก็บในตัวแปรชุดแบบ Dynamic Allocation ชนิด Vector ก่อนจะไปเรียงลำดับโดยใช้วิธีเรียงลำดับแบบ Quick Sort โดยเรียงจากมากไปหาน้อย

ส่วนแสดงผลลัพธ์

จะแสดงผลลัพธ์ที่ได้จากการสืบค้น โดยจะแสดงถึงค่าที่ใช้ในการสืบค้น จำนวนเอกสารผลลัพธ์ที่ได้จากการสืบค้น และเวลาที่ใช้ในการสืบค้น

ซึ่งในแต่ละเอกสาร จะมีการแสดงรายละเอียดเกี่ยวกับเอกสารดังนี้

- ชื่อเรื่อง (Title)
- คำสำคัญ (Keywords) คือ Index terms
- บทคัดย่อ (Abstract)
- ชื่อผู้แต่ง (Author)
- ตัวเลือกเพื่อดูเอกสาร (Download)

โดยจะแสดงผลเรียงตามลำดับที่ได้ทำการเรียงลำดับไว้ในขั้นตอนการเรียงลำดับผลลัพธ์

ในกรณีที่ตัวเลือกเพื่อดูเอกสารนั้น เนื่องจากเอกสารที่เราจัดเก็บในรูปแบบของแฟ้มเอกสาร PDF จากโปรแกรม Adobe Acrobat ดังนั้นหากคอมพิวเตอร์ของผู้สืบค้นมีโปรแกรม Adobe Acrobat ติดตั้งอยู่ก็จะสามารถแสดงผลได้ทันที และผู้ใช้จะสามารถจัดเก็บเอกสารนั้นไว้ที่คอมพิวเตอร์ของตนเองได้ โดยเลือกคำสั่ง Save ของโปรแกรม

ส่วนจัดการเอกสาร (บรรณารักษ์)

ส่วนเพิ่มเอกสาร

ใช้ในการเพิ่มเอกสารใหม่เข้าสู่ระบบ โดยจะมีลักษณะเป็นแบบฟอร์ม ซึ่งบรรณารักษ์จะกรอกรายละเอียดต่างๆเกี่ยวกับเอกสารลงไป และรายละเอียดเหล่านี้จะถูกนำไปเก็บไว้ในตาราง Paper ยกเว้น Index terms ที่จะถูกนำไปเก็บไว้ในตาราง PaperKey และชื่อผู้แต่งที่จะถูกนำไปเก็บไว้ในตาราง Author นอกจากนี้ค่า Index terms ที่ได้มาใหม่จะถูกนำไปคำนวณปรับค่าความสัมพันธ์ระหว่างค่าในตาราง Relation ด้วย

ส่วนเปลี่ยนแปลงเอกสาร

มีหน้าที่นำข้อมูลเกี่ยวกับเอกสารออกมาให้บรรณารักษ์ตรวจสอบ ลบเอกสาร หรือแก้ไขข้อมูลต่างๆ แบ่งออกเป็น 2 ส่วน ได้แก่

ส่วนค้นหาเอกสาร มีลักษณะคล้ายกับการสืบค้นตามเงื่อนไขอื่นๆที่ไม่ใช่ Index terms ของผู้ใช้ปกติ โดยมีส่วนเพิ่มขึ้นมาคือ การค้นหาบรรณารหัสเอกสารของระบบที่ระบบจะสร้างขึ้นให้เอกสารใหม่ ทุกครั้งที่มีการเพิ่มเอกสาร ซึ่งจะมีการค้นหาโดย

- การค้นหาแบบธรรมดา ไม่มีการคำนวณเพิ่มเติม
- เงื่อนไขทั้งหมดจะถูกนำมาคิดรวมกัน (AND)
- เป็นการค้นหาแบบ Pattern Matching

ซึ่งผลลัพธ์ที่ได้จากการค้นหาจะถูกนำมาแสดงผลต่อลงมาจากแบบฟอร์มที่ใช้ ซึ่งบรรณารักษ์จะสามารถเรียงลำดับการแสดงผลได้ โดยการเลือกที่ชื่อ Field ที่ต้องการให้ทำการเรียงลำดับ โดยที่การเรียงลำดับนี้จะใช้วิธีการส่งคำสั่ง SQL เข้าเดิมอีกครั้ง โดยเพิ่มส่วนของการจัดลำดับ (ORDER BY) เข้าไป

จากผลลัพธ์ที่ได้จากการค้นหา บรรณารักษ์สามารถแก้ไขเปลี่ยนแปลงเอกสารได้ โดยการเลือกที่ชื่อเรื่องของเอกสาร ที่ต้องการลบหรือแก้ไข ซึ่งจะแสดงข้อมูลของเอกสารให้บรรณารักษ์แก้ไขเปลี่ยนแปลงต่อไป

ส่วนการลบเอกสาร เป็นการลบเอกสาร รวมถึงข้อมูลต่างๆของเอกสารนั้นๆ ออกจากระบบ โดยเมื่อเลือกที่จะลบเอกสารนั้นๆแล้ว จะเกิดการลบข้อมูลเอกสาร ชื่อผู้แต่ง และ Index terms ออกจากตารางต่างๆ รวมถึงการปรับเปลี่ยนค่าในตาราง Relation ใหม่

ส่วนการแก้ไขเอกสาร บรรณารักษ์สามารถดูข้อมูลที่มีอยู่เดิม และตัดสินใจแก้ไขข้อมูลต่างๆก่อนจะบันทึกได้ ซึ่งเมื่อแก้ไขข้อมูลต่างๆแล้ว จะมีการเปลี่ยนแปลงข้อมูลที่แก้ไขนั้นในตารางที่มีข้อมูลนั้นๆอยู่ในกรณีที่มีการแก้ไข Index terms ก็ต้องมีการปรับค่าในตาราง Relation ด้วย และนอกจากนี้บรรณารักษ์ยังสามารถ Upload เอกสารใหม่ขึ้นไปแทนที่ของเดิมได้ ซึ่งเอกสารใหม่นั้นจะมีชื่อเดิมหรือมีชื่อใหม่ก็ได้

ส่วนจัดเก็บเอกสาร (Upload)

เป็นส่วนที่จัดการเกี่ยวกับการดึงแฟ้มเอกสารจากคอมพิวเตอร์ของผู้ใช้ ไปเก็บไว้บน Server ในตำแหน่ง Directory ที่กำหนดไว้ ซึ่งส่วนนี้จะมียู่ทั้งในส่วนของการเพิ่มเอกสาร แก้ไขเอกสาร และ Automatic Indexing

ตามปกติแล้วเมื่อเราใช้ INPUT แบบ FILE ใน HTML(HyperText Markup Language) และเรากำหนดรูปแบบการส่งข้อมูลกลับของ FORM ให้เป็นแบบ POST แล้ว Browser จะทำหน้าที่ส่งข้อมูลในแฟ้มนั้นๆมาให้อยู่แล้ว แต่ข้อมูลจะอยู่ในรูปแบบเฉพาะ ซึ่งส่วนนี้จะทำหน้าที่อ่านข้อมูลนั้น และนำมาสร้างเป็นแฟ้มข้อมูลใหม่ที่ฝั่ง Server

```
<FORM ACTION="http://www.mysearch.com/librarian/chkupdate.html"
METHOD="POST" ENCTYPE="multipart/form-data">
<INPUT TYPE="file" NAME="file" SIZE="35" VALUE="">
</FORM>
```

การใช้ INPUT แบบ FILE ใน HTML

```
-----7d23c11e04
Content-Disposition: form-data; name="file"; filename="C:\Test.txt"
Content-Type: text/plain

This is a test.
-----7d23c11e04
Content-Disposition: form-data; name="Submit"

Submit
-----7d23c11e04--
```

ผลลัพธ์ที่ได้จาก Browser ในกรณีแฟ้มชื่อ Test.txt มีข้อมูล This is a test.

ซึ่งในส่วนนี้เราจำเป็นต้องใช้บริการจาก Web browser เนื่องจากเราไม่สามารถเปิดแฟ้มข้อมูลในคอมพิวเตอร์ของผู้อื่นได้ เนื่องจากมาตรฐานด้านการรักษาความปลอดภัย แต่เราสามารถสร้างแฟ้มใหม่ที่ Server ได้ เพราะ JSP ของเราทำงานอยู่บน Server จึงสามารถกำหนดสิทธิให้สามารถอ่าน เขียน แฟ้มข้อมูลได้

ส่วนจัดเก็บเอกสารนี้ มีลักษณะเป็น Java Bean ชื่อ jspSmartUpload ทำงานร่วมกับ JSP โดยส่วนของ Bean จะทำหน้าที่อ่านข้อมูลและส่งข้อมูลในแฟ้มข้อมูลในรูปแบบของ String กลับมาให้ JSP ทำหน้าเขียนแฟ้มข้อมูล

ส่วนที่เป็น Java Bean มีโครงสร้าง ดังนี้

Package : com.jspSmart.Upload.*	
Class SmartUpload	
Void	initialize (javax.servlet.jsp.PageContext pageContext) กำหนดค่าเริ่มต้นในการทำงาน
void	setTotalMaxFileSize (long totalMaxFileSize) กำหนดค่าขนาดแฟ้มรวมสูงสุดที่รับได้

Package : com.jspSmart.Upload.*	
Class SmartUpload (continue)	
void	upload() รับข้อมูลจากฟอร์ม
com.jspSmart.Upload.Files	getFiles() ส่งข้อมูลที่ได้กลับมาในรูป Class Files
Class Files	
int	getCount() ให้ค่าจำนวนแฟ้มข้อมูลทั้งหมดที่ได้รับมา
com.jspSmart.Upload.File	getFile(int index) ส่ง File ตำแหน่งที่ระบุออกมา
Class File	
int	getSize() ให้ค่าความยาวของข้อมูลในแฟ้มข้อมูล
java.lang.String	getContentString() ส่งข้อมูลในแฟ้มข้อมูลออกมา
java.lang.String	getFileName() ให้ชื่อแฟ้มข้อมูลออกมา
boolean	isMissing() ตรวจสอบว่าแฟ้มข้อมูลมีอยู่จริงหรือไม่ (กรณีแฟ้มข้อมูลไม่มีอยู่จริงบนฝั่งผู้ใช้)

ส่วน Automatic Indexing

เป็นทางเลือกที่ผู้ใช้อาจจะใช้หรือไม่ใช้ก็ได้ ในกรณีที่เอกสารมี Index terms มาให้อยู่แล้ว ก็ไม่มีความจำเป็นต้องใช้ส่วนนี้ ปกติแล้วส่วนนี้จะทำหน้าที่อ่านแฟ้มข้อมูลที่เป็น Text file ที่ได้รับการแปลงมาจาก PDF File ของเอกสาร แล้วนำมาคำนวณหาว่าคำใดน่าจะเป็น Index terms ของเอกสาร ซึ่งมีขั้นตอนการทำงานแบ่งออกเป็น 2 ขั้นตอนใหญ่ๆ ดังนี้

1. การเก็บสถิติของคำ มีขั้นตอน ดังนี้
 - นำข้อมูลทั้งหมดในแฟ้มข้อมูลไปผ่านขั้นตอนการทำ Text Operations
 - นับจำนวนคำที่ได้จากการทำ Text Operations และกำหนดช่วงที่ต้องการแบ่งเอกสาร เพื่อคำนวณค่าน้ำหนัก (ปกติใช้ 1 : 3 : 1)
 - พิจารณาคำนับนั้นๆอยู่ในช่วงใดของเอกสาร แล้วเก็บค่าไว้
 - นำคำมาเปรียบเทียบหาคำซ้ำ หากซ้ำนำค่าที่เก็บไว้มารวมกัน
- ส่วนนี้มีลักษณะเป็น Java Bean มีโครงสร้าง ดังนี้

Package : NN.*	
Class NWord	
void	AddData(java.lang.String reference,java.lang.String word,int position) เพิ่มข้อมูล โดยจะตรวจสอบว่าซ้ำหรือไม่ ถ้าซ้ำจะรวมเป็นข้อมูลเดียว

Package : NN.*	
Class Nword (continue)	
int	GetSize() ให้จำนวนข้อมูลทั้งหมด
java.lang.String	GetWord(int index) ให้ข้อมูลคำในตำแหน่งที่กำหนด
java.lang.String	GetRef(int index) ให้ข้อมูลคำอ้างอิงในตำแหน่งที่กำหนด
int	GetUFR(int index) ให้ข้อมูลจำนวนคำที่พบในส่วนต้นของเอกสาร(1/5) ในตำแหน่งที่กำหนด
int	GetUMR(int index) ให้ข้อมูลจำนวนคำที่พบในส่วนกลางของเอกสาร(3/5) ในตำแหน่งที่กำหนด
int	GetULR(int index) ให้ข้อมูลจำนวนคำที่พบในส่วนท้ายของเอกสาร(1/5) ในตำแหน่งที่กำหนด
void	SetWC(int wordcount) เก็บจำนวนคำก่อนเปรียบเทียบ
int	GetWC() ให้จำนวนคำก่อนเปรียบเทียบ

2. การคำนวณ ในกรณีนี้เราจะกำหนดค่าความสำคัญในส่วนต่างๆของเอกสารเป็นค่าคงที่ ซึ่งมีรายละเอียดดังนี้

ส่วนต้นของเอกสาร (1/5)	3.0
ส่วนกลางของเอกสาร (3/5)	1/3
ส่วนท้ายของเอกสาร (1/5)	2.0

เราจะนำค่าเหล่านี้มาคูณกับจำนวนคำที่อยู่ในส่วนต่างๆ แล้วนำค่าทุกส่วนที่ได้มารวมกันเป็นค่าของคำนั้นๆ จากนั้นนำค่าทั้งหมดมาเรียงลำดับจากมากไปน้อย แล้วนำจำนวนคำก่อนเปรียบเทียบทั้งหมดของเอกสารมาหาตำแหน่งที่เราต้องการ(ปกติใช้ 1/50) เพื่อทำการแบ่งส่วนที่คาดว่าจะเป็ Index terms ซึ่งจะเป็ส่วนที่มีอยู่ในลำดับที่ 1 ถึงตำแหน่งที่กำหนด

ส่วนการเข้าสู่ระบบ (log in)

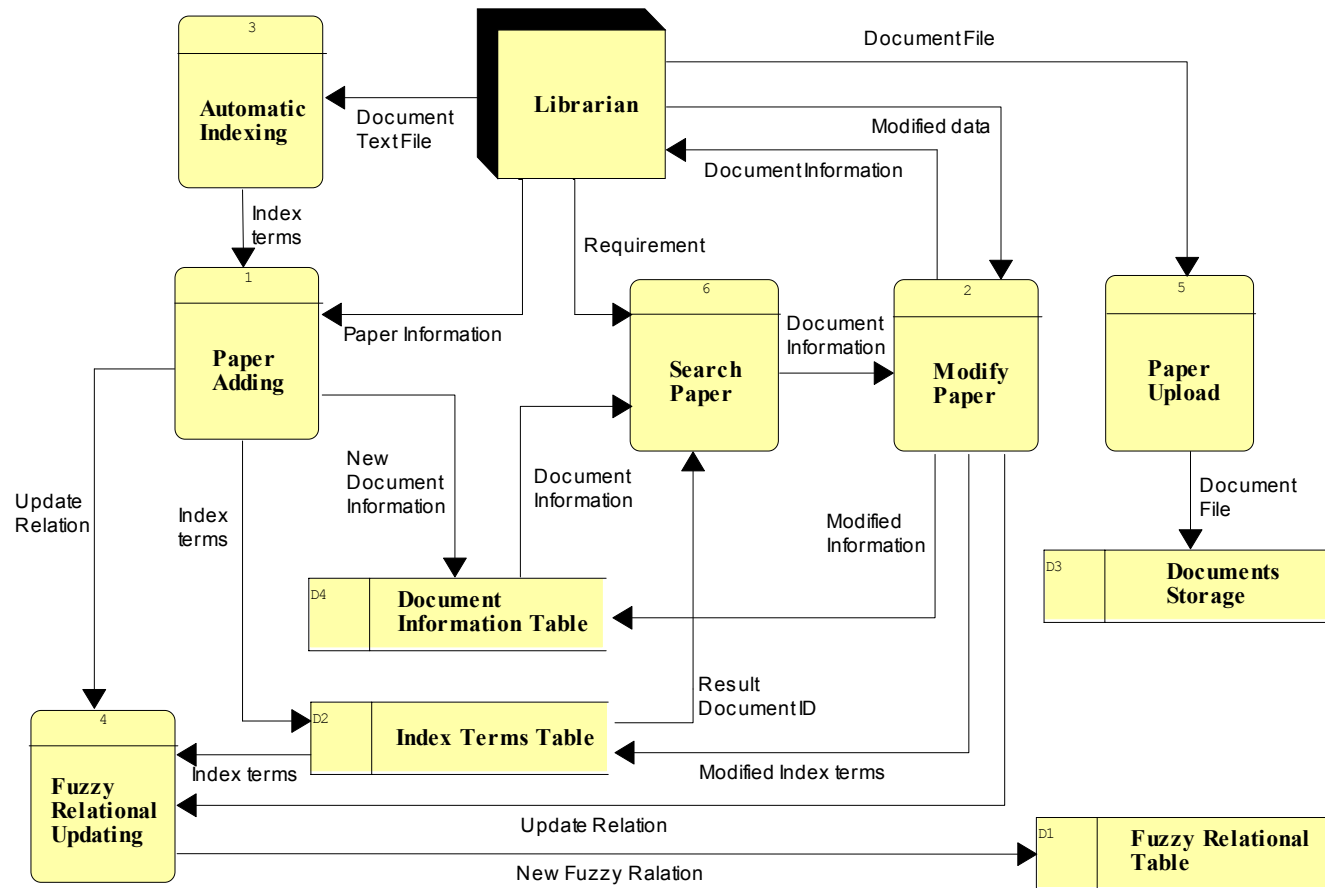
เนื่องจากผู้ใ้ทั่วไปไม่ควรที่จะเปลี่ยนแปลงแก้ไขข้อมูลเกี่ยวกับเอกสารใดๆได้ ดังนั้นส่วนนี้จึงต้องมีการยืนยันบุคคลที่เข้าใช้ (Authentication) ซึ่งเราใ้การใส่ชื่อผู้ใ้และรหัสผ่านที่ถูกต้อง โดยจะมีการตรวจสอบกับข้อมูลในฐานข้อมูล เพื่อยืนยัน เมื่อยืนยันแล้วจะเก็บเป็น Sessions ไว้เพื่อสามารถเข้าสู่หน้า Web page ต่างๆต่อไปได้ (อาจใ้ SSL(Secure Sockets Layer) เพื่อเพิ่มความปลอดภัย)

ส่วนการออกจากระบบ (log out)

เป็นส่วนที่ทำหน้าที่ลบ Sessions ทั้งหมด เมื่อจบการทำงาน

ส่วนการเปลี่ยนรหัสผ่าน

เพื่อให้บรรณารักษ์สามารถเปลี่ยนแปลงรหัสผ่านได้ โดยต้องใ้รหัสผ่านเดิมประกอบ



รูปแสดง *Data Flow Diagram* ของส่วนจัดการเอกสาร (บรรณารักษ์)

ส่วนจัดการข้อความ (Text Operations)

ส่วนแบ่งคำ

เป็นส่วนที่ทำหน้าที่แบ่งข้อความออกเป็นคำ ซึ่งมีขั้นตอนการทำงาน ดังนี้

1. แบ่งคำ โดยใช้เครื่องหมายต่างๆเป็นตัวแบ่ง ยกเว้น – และ . ซึ่งต้องไปพิจารณาเงื่อนไขข้อ 2
2. กรณี – จะคิดเป็นคำเดียวกันต่อเมื่อ
 - มีบางกลุ่มในกลุ่มที่เชื่อมกันด้วย – เป็น ตัวอักษรพิมพ์ใหญ่ทั้งหมด เช่น N-Layer, N-P-N
 - มีบางกลุ่มในกลุ่มที่เชื่อมกันด้วย – เป็นตัวเลขทั้งหมด เช่น B-79, M-16
3. กรณี . จะคิดเป็นคำเดียวกันก็ต่อเมื่อ เป็นตัวอักษรพิมพ์ใหญ่ 1 ตัว สลับกับ . 1 จุด เช่น W.T.O. , W.H.O.
4. สำหรับกลุ่มของตัวเลขที่ถูกแบ่งจากข้อ 1 โดยไม่มีตัวอักษรอยู่ในกลุ่มเลยจะถูกกำจัด ส่วนที่มีตัวอักษรอยู่ในกลุ่มจะคงไว้ เช่น Direct3D, RFC2345
5. ตัวอักษรทั้งหมดจะถูกทำให้เป็นพิมพ์เล็ก (Lower Case) เพราะโดยปกติแล้ว Case ของตัวอักษรจะไม่มีผลสำคัญในการสืบค้นข้อมูล

ซึ่งในขั้นตอนที่ 1 – 4 เราสามารถรวมเข้าด้วยกัน โดยอาศัยหลักการของ State machine แบบ Pushdown Automata ได้

ส่วนการจัด Stopwords

คือ การจัดคำทั่วไปที่ไม่มีความสำคัญหรือเกี่ยวข้องกับเนื้อหาภายในเอกสาร โดยใช้วิธีการเปรียบเทียบคำกับฐานข้อมูล Stopwords หากคำใดตรงกับคำในฐานข้อมูลก็จะกำจัดคำนั้นทิ้ง

วิธีการที่ใช้ในการเปรียบเทียบแบ่งออกเป็น 2 วิธี

- ใช้ SQL ในการเปรียบเทียบ โดยใช้การนับคำที่ตรงกัน ซึ่งสะดวก แต่ใช้เวลามาก
(Select Count(*) from Stopword where Word='คำ')
- นำข้อมูลทั้งหมดในฐานข้อมูลขึ้นมาไว้บน Memory ก่อน แล้วจึงทำการเปรียบเทียบ
ซึ่งในกรณีที่มีคำที่ต้องการเปรียบเทียบมาก จะเป็นวิธีที่เร็วกว่า

ส่วนกำจัด Suffix

หรือส่วนที่ทำ Stemming เพื่อให้ได้รากศัพท์ กรณีนี้เราจะใช้ Porter's Algorithm ในการกำจัด Suffix โดยจะแบ่งการทำงานออกเป็น 5 ขั้นตอนใหญ่ๆ ซึ่งรายละเอียดเกี่ยวกับ Porter's Algorithm นี้สามารถศึกษาได้จากภาคผนวกของปริญาานิพนธ์

เนื่องจากการทำ Porter's Algorithm นี้ ผลลัพธ์ที่ได้อาจไม่ตรงกับรากศัพท์ที่แท้จริง โดยอาจผิดเพี้ยนไปบ้าง เช่น browsing และ browser จะได้ผลลัพธ์เป็น brows ซึ่งไม่สื่อความหมายต่อผู้ใช้งาน จึงจำเป็นต้องมีการเก็บคำอ้างอิง หรือ Reference word ไว้ทุกครั้ง ในกรณีที่ต้องการแสดงผลให้ผู้ใช้งานรับทราบ และคำที่จะนำมาเปรียบเทียบกับฐานข้อมูลควรผ่าน Porter's Algorithm ทุกครั้ง เพื่อจะได้ถูกต้องตรงกัน คล้ายกับการทำให้เป็น Normal form

ส่วน Thesaurus

หรือส่วนที่เพิ่มคำเหมือนเข้าไปในการค้นหา เพื่อให้ผลลัพธ์ที่ถูกต้องมากขึ้น เป็นส่วนที่เป็น Static Thesaurus ซึ่งเก็บไว้ในฐานข้อมูล เราจะใช้ช่วยเฉพาะในส่วนของการสืบค้นเท่านั้น โดยเลือกคำที่เป็น Synonym ของคำที่รับเข้ามาทำการสืบค้น

สำหรับการกำหนดค่าในเชิง Fuzzy นั้น เราจะไม่กำหนดค่าให้เป็น 1.0 เลย เพราะไม่ใช่ค่าที่ผู้ใช้งานต้องการสืบค้นโดยตรง แต่เราจะกำหนดค่าให้มีค่าความสัมพันธ์ที่สูง (ปกติใช้ประมาณ 0.9) ซึ่งตามความเป็นจริงแล้วค่านี้ไม่ควรเท่ากันหมด แต่เนื่องจากไม่มีวิธีที่จะคำนวณค่าออกมาได้ เพราะต้องใช้ความรู้เกี่ยวกับภาษา นอกจากนี้ยังขึ้นอยู่กับมุมมอง และบริบทของคำในประโยค จึงจำเป็นต้องกำหนดให้เป็นค่าคงที่ไว้

โครงสร้างของส่วนจัดการข้อความ

เนื่องจากส่วนจัดการข้อความเป็นส่วนที่มีการใช้มาก ในหลายๆส่วนของระบบ เราจึงรวมฟังก์ชันในการจัดการข้อความส่วนใหญ่ (ยกเว้นส่วน Thesaurus ซึ่งใช้เฉพาะในส่วนสืบค้น) ไว้ในลักษณะของ Java Bean ซึ่งมีโครงสร้าง ดังนี้

Package : Text.*	
Class TextOp	
java.lang.String	Porter (java.lang.String word) กำจัด Suffix ของคำ โดยใช้ Porter's Algorithm
java.lang.String	Remove (java.lang.String str, char rmv) กำจัดตัวอักษรที่ระบุออกจากข้อความที่กำหนด
java.lang.String	Gword (java.lang.String str, int position) แบ่งคำ 1 คำออกจากข้อความ เริ่มที่ตำแหน่งที่กำหนด
int	getNext () ให้ค่าตำแหน่งสุดท้ายของคำที่ถูกแบ่งออกจากข้อความ
java.sql.Connection	Connect (java.lang.String driver, java.lang.String url, java.lang.String username, java.lang.String password) ติดต่อกับฐานข้อมูลที่กำหนด
int	SSW (java.lang.String word, java.sql.Connection connection) ตรวจสอบว่าคำที่ให้ตรงกับข้อมูลในตาราง Stopwords หรือไม่
java.util.Vector	SSW (java.sql.Connection connection) คืน Vector ที่เก็บคำทุกคำในตาราง Stopwords
boolean	ChkSW (java.lang.String word, java.util.Vector stopwords) ตรวจสอบว่าคำที่ให้ตรงกับข้อมูลใน Vector ที่ได้รับมาหรือไม่
java.lang.String	Close (java.util.Connection connection) ปิดการติดต่อกับฐานข้อมูล ส่งค่าความผิดพลาดกลับ ถ้าไม่มีส่ง null

การออกแบบฐานข้อมูล

ฐานข้อมูลถือเป็นส่วนสำคัญในระบบการสืบค้น โดยฐานข้อมูลที่ใช้ในระบบของเราจะแบ่งออกได้เป็น 2 ประเภท ได้แก่

- ฐานข้อมูลที่ใช้อ้างอิงเพียงอย่างเดียว ฐานข้อมูลประเภทนี้มักไม่มีการเปลี่ยนแปลงแก้ไข เป็นเหมือนกับค่าคงที่ ที่ระบบใช้อ้างอิง เช่น Stopwords list, Thesaurus เป็นต้น
- ฐานข้อมูลทั่วไปที่มีการเปลี่ยนแปลงแก้ไข ส่วนใหญ่เกี่ยวข้องกับการทำงานของระบบ แม้จะมีการเปลี่ยนแปลงแก้ไข แต่ก็ถือว่าน้อยมาก เมื่อเทียบกับการสืบค้นข้อมูล ส่วนใหญ่จะเกี่ยวกับเอกสาร เช่น Paper , Author , Relation เป็นต้น

ฐานข้อมูลทั่วไป

PAPER

← p,k →

Pnumber	Pname	Abstract	IndexRef	Location	Filesize
---------	-------	----------	----------	----------	----------

ตาราง PAPER ทำหน้าที่เก็บข้อมูลของเอกสารแต่ละเอกสาร

- Pnumber เป็นคีย์หลักทำหน้าที่ในการอ้างอิงเอกสาร
- Pname ชื่อหัวเรื่องของเอกสาร
- Abstract บทคัดย่อของเอกสาร
- IndexRef ใช้อ้างอิงเพื่อใช้ในการแก้ไข เพราะ index ที่ใช้ในการหาค่าความสัมพันธ์จะอยู่ในรูปแบบที่เป็นรากศัพท์ ดังนั้นผู้แก้ไขจะไม่ทราบที่มาของ index term นั้น
- Location เก็บตำแหน่งของแฟ้มเอกสาร
- Filesize เก็บขนาดของแฟ้มเอกสาร

AUTHOR

← p,k. →

← f,k →

Pnumber	Authurname
---------	------------

ตาราง AUTHOR ทำหน้าที่เก็บข้อมูลผู้แต่งเนื่องจากเอกสารแต่ละฉบับ อาจมีผู้แต่งได้มากกว่า 1 ท่าน

- Pnumber หมายเลขกำกับเอกสารเพื่อใช้ในการค้นหา
- Authurname ชื่อผู้แต่ง

PAPERKEY

← p,k. →

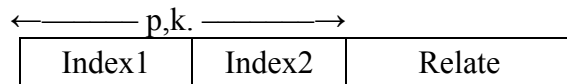
← f,k →

Pnumber	Keyword
---------	---------

ตาราง PAPERKEY ใช้สำหรับเก็บ index term ของแต่ละเอกสารนั้น

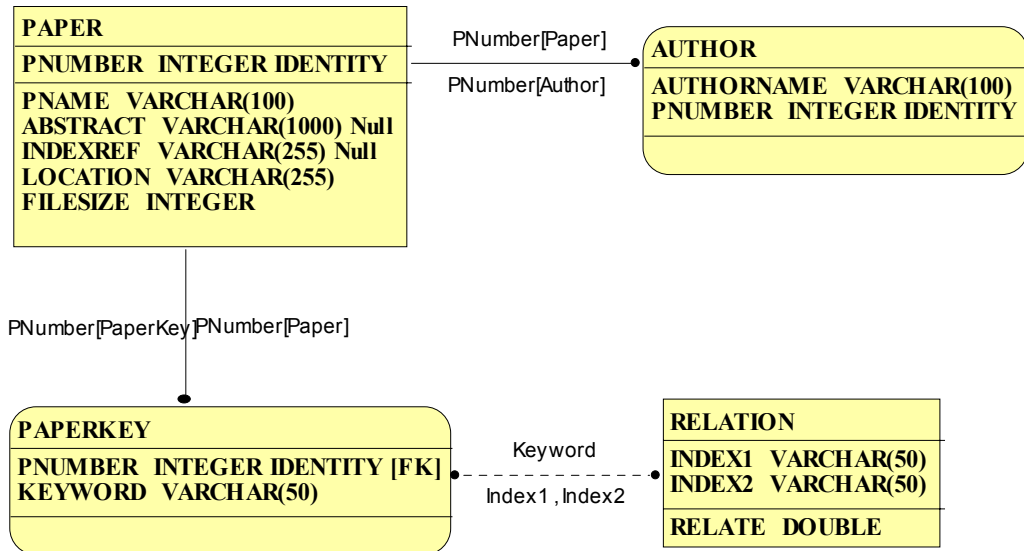
- Pnumber หมายเลขกำกับเอกสารเพื่อใช้ในการค้นหา
- Keyword index term

RELATION



ตาราง RELATION ใช้สำหรับเก็บคู่ความสัมพันธ์ของ index term

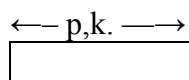
- Index1, Index2 คู่ Index term
- Relate ค่าความสัมพันธ์มีค่าอยู่ระหว่าง 0-1



ER Diagram ของฐานข้อมูลทั่วไป

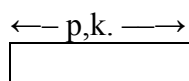
ฐานข้อมูลอ้างอิง

STOPWORD



ตาราง STOPWORD ใช้สำหรับเก็บ Stopwords เพื่อใช้ในการ Text Operations

THESAURUS



ตาราง THESAURUS ใช้สำหรับเก็บคำเหมือน คำคล้าย เพื่อเพิ่มความละเอียดในการสืบค้น